CCA-F Certification Prep Week 3 · Day 12

Context Management & Reliability

1. Why reliability matters at scale

A Claude integration that works 95% of the time fails 1 in 20 requests. At 10,000 requests/day that's 500 failures daily. Reliability engineering is about designing systems that handle the 5% gracefully — through confidence calibration, self-review, and multi-agent verification patterns.

2. Confidence calibration

Confidence calibration means explicitly instructing Claude to report its uncertainty alongside its answer. Without this, Claude states uncertain answers with the same tone as certain ones — making it impossible to know when to trust the output.

Without calibration ✗

```
Q: What was our Q3 revenue?
A: Your Q3 revenue was $4.2M.

(Claude sounds certain — may be hallucination)
```

With calibration ✓

```
Q: What was our Q3 revenue?
A: {
  "answer": "$4.2M",
  "confidence": "medium",
  "reasoning": "Based on Q2 data in
    context. Q3 not found.",
  "recommend_verify": true
}
```

Routing by confidence level

- High confidence (directly in context) → return to user automatically.
- Medium confidence (inferred from context) → flag for human review.
- Low confidence (not in context, from general knowledge) → escalate to human.
- Never auto-act on low-confidence outputs — always route to human review.
- 'I don't know' is a valid and important output. Instruct Claude to use it.

3. Self-review — the two-pass pattern

Self-review is a two-pass pattern where Claude first generates an answer, then critiques its own output in a second pass before returning the final result. Improves accuracy by 15–25% on complex tasks.

```
# Self-review prompt pattern
Step 1: Answer this question: {{question}}

Step 2: Review your answer above. Check:
- Is every claim supported by the context provided?
- Are there any logical errors in your reasoning?
- Is anything missing or overstated?

Step 3: Output your final corrected answer as JSON:
{"answer": string, "corrections_made": [], "confidence":
"high"|"medium"|"low"}
```

Self-review — key facts

- Two-pass: generate first, then critique the same output.
- Improves accuracy by approximately 15-25% on complex reasoning tasks.
- Limitation: both passes share the same reasoning errors and blind spots.
- The critique pass should be adversarial — instruct it to actively find flaws.
- Use self-review for moderate-stakes, cost-sensitive tasks.

4. Multi-instance review

Multi-instance review runs the same task through multiple independent Claude instances and compares results. Agreement = high confidence. Disagreement = flag for human review. Independent instances don't share the same reasoning errors.

Multi-instance aggregation rules

- All instances agree → high confidence → auto-return result.
- Partial disagreement → medium confidence → flag for human review.
- Full disagreement → low confidence → escalate to human.
- High-stakes domain (medical, legal, financial) → escalate on ANY disagreement.
- Majority vote is NOT a substitute for human review in high-stakes decisions.
- Response order is irrelevant — instances run in parallel, there is no 'first' answer.

Self-review vs multi-instance — when to use which

	Self-review	Multi-instance review
How it works	One Claude, two passes	Multiple Claude instances, one pass each
Cost	~2x tokens	~3x tokens or more
Accuracy gain	~15-25% improvement	Higher — no shared blind spots
Limitation	Same error in both passes	Token cost, latency
Use when	Moderate stakes, cost-sensitive	High stakes, accuracy-critical
Exam trigger	'Improve without human review'	'Highest accuracy', 'critical decision'

5. Long-context strategies

Long-context tasks require specific strategies to maintain accuracy. Claude's accuracy degrades for content positioned in the middle of a long context — the 'lost in the middle' problem.

The lost-in-the-middle problem

- Claude tends to focus on content at the beginning and end of long contexts.
- Information in the middle of a long document is systematically underweighted.
- Diagnostic signature: edge content correct, middle content missed.
- This is a known attention pattern — not a bug, but a design constraint to engineer around.

Fix 1 — Strategic positioning

- Place the most important content at the beginning OR end of the context.
- Never bury critical constraints or data in the middle of a long document.
- System prompt instructions at the top, reinforced at the bottom.

Fix 2 — Chunk and summarise

- Break large documents into sections (e.g. 10 pages each).
- Process each chunk independently in its own context window.
- Synthesise chunk summaries into a final result.
- Each chunk's content is now at the beginning/end of its own context — maximum attention.

Fix 3 — Repeat critical instructions

- For very long prompts, repeat key constraints at both start and end.
- Example: 'Output only JSON. [... long document ...] Reminder: output only JSON.'
- Instruction repetition combats the attention dropoff in the middle.

6. Hallucination mitigation toolkit

Claude can produce confident-sounding incorrect outputs. The exam tests which techniques reduce this risk.

Ground outputs in provided context

- Answer ONLY using information from the document below.
- If the answer is not in the document, say 'Not found in context.'
- Do not use your general knowledge.
- This is the most effective single technique for factual accuracy.

Citation requirement

- Require Claude to cite the exact sentence supporting each claim.
- Format: [claim] (Source: paragraph N, sentence M)
- Claude cannot hallucinate a citation without it being verifiable.
- Uncited claims are flagged for review.

Confidence scoring per claim

- Require a confidence score for every individual claim, not just the overall answer.
- Low-confidence claims trigger targeted human review of just those claims.
- More granular than overall confidence scoring.

Adversarial critique in self-review

- In the self-review critique pass, instruct Claude to actively find errors.
- Prompt: 'Your job is to find every flaw. Be critical and skeptical. What could be wrong?'
- Adversarial framing produces more thorough critiques than neutral review.

7. Exam-critical summary table

Concept	Key rule
Confidence calibration	Report uncertainty alongside answer. Route: high=auto, medium=flag, low=escalate.
Self-review	Two-pass: generate then critique. ~15-25% accuracy gain. Same blind spots in both passes.
Multi-instance review	Multiple independent instances. Higher accuracy. Disagreement = human review.
Lost in the middle	Accuracy drops for middle content. Fix: strategic positioning + chunking.
Hallucination mitigation	Ground in context + citation requirement + confidence scoring + adversarial critique.
Routing by confidence	Never auto-act on low-confidence. High=auto, medium=flag, low=escalate.
High-stakes domains	Medical, legal, financial: escalate on ANY disagreement. Never majority vote.
Instruction repetition	Repeat key constraints at start AND end of long prompts to combat attention dropoff.

8. Quick recall — 3 things to remember

1. **Self-review doesn't fix shared blind spots.**
   Both passes share the same reasoning error. For high-stakes decisions, use multi-instance review instead.

2. **Lost in the middle is real.**
   Claude's accuracy drops for information in the middle of long contexts. Always position critical content at the start or end.

3. **'I don't know' is a valid output.**
   Instruct Claude to express uncertainty rather than guess. Low-confidence answers must be routed to human review, not auto-acted upon.

9. Practice questions & answers

All three questions answered correctly — perfect score!

Q1. A team deploys Claude to automatically approve or reject loan applications using a single instance returning results immediately. After weeks, they find Claude has been confidently approving applications it should have rejected, with no way to detect uncertain decisions. What reliability pattern should they implement?

A) Validation retry loop — retry until Claude produces a consistent answer

✓ B) Confidence calibration + multi-instance review — require uncertainty reporting and cross-check high-stakes decisions

C) Few-shot prompting — provide more correct loan decision examples

D) Chain-of-thought — force step-by-step reasoning before deciding

Explanation:

- B is correct: Confidence calibration solves the first problem — uncertain decisions invisible. Requiring confidence field (high/medium/low) enables routing. Multi-instance review solves accuracy — independent instances catch errors self-review misses. Disagreement = human review.
- A is wrong: Retry loops fix format problems (invalid JSON). They don't fix reasoning accuracy.
- C is wrong: Few-shot helps consistency but doesn't add uncertainty detection or cross-checking.
- D is wrong: CoT improves reasoning quality but still one answer from one instance with no uncertainty signal.

Q2. A Claude agent processes a 50-page legal contract for liability clauses. The lawyer finds clauses from pages 18-32 were completely missed, while first and last pages were correct. What failure mode is this and what is the fix?

A) Context overflow — document exceeded context window. Fix: larger model

B) Goal drift — agent lost its objective mid-document. Fix: pin the goal

✓ C) Lost in the middle — accuracy drops for middle-positioned content. Fix: chunk the document

D) Prompt injection — contract contained instructions redirecting Claude

Explanation:

- C is correct: Textbook lost-in-the-middle. Pages 1-5 and last pages correct (edges), pages 18-32 missed (middle). Fix: chunk into sections, process each independently, synthesise.
- A is wrong: Context overflow would fail entirely or truncate — not selectively miss the middle.
- B is wrong: Goal drift = goal lost from context overflow. Here Claude maintained the goal, just missed middle content. Different failure mode.
- D is wrong: Prompt injection produces wrong output types — not systematic middle-content gaps.

Q3. Three independent Claude instances analyse the same medical case. Results: Instance A: {diagnosis: 'Type 2 Diabetes', confidence: 'high'}, Instance B: {diagnosis: 'Type 2 Diabetes', confidence: 'high'}, Instance C: {diagnosis: 'Pre-diabetes', confidence: 'medium'}. What should the aggregator do?

A) Accept majority vote — 2 out of 3 agree on Type 2 Diabetes

B) Accept Instance A's answer — it responded first

✓ C) Flag the disagreement and escalate to a human doctor — disagreement signals uncertainty

D) Run a fourth instance as tiebreaker

Explanation:

- C is correct: Disagreement between instances = uncertainty. In medical context — high-stakes, irreversible — any disagreement must trigger human review. Pre-diabetes vs Type 2 Diabetes is clinically significant and affects treatment.
- A is wrong: Majority vote is dangerous for high-stakes decisions. Two instances sharing the same error always outvote a correct minority. Agreement ≠ correctness.
- B is wrong: Response order is irrelevant. Instances run in parallel — there is no 'first'.
- D is wrong: A fourth Claude instance doesn't resolve genuine clinical ambiguity. Human expertise needed.

Day 12 scorecard

3 / 3

Perfect score! Reliability instincts are excellent.

Topic	Result
Confidence calibration + multi-instance for loan decisions	✓ Correct
Lost in the middle — chunking fix	✓ Correct
Multi-instance disagreement → human escalation	✓ Correct

Multi-instance aggregation rule — locked in

- All instances agree → high confidence → auto-return result.
- Partial disagreement → medium confidence → flag for human review.
- Full disagreement → low confidence → escalate to human.
- High-stakes domain (medical, legal, financial) → escalate on ANY disagreement.
- Majority vote is NOT a substitute for human review in high-stakes decisions.
- Response order is irrelevant — parallel execution has no meaningful 'first'.

Next up: Day 13	Week 3 Friday quiz — 10 questions covering prompt engineering + context management (35% of exam).